Imports

```
import pandas as pd
import geopandas as gpd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, DBSCAN
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

Load & Inspect Data

```
gdf = gpd.read_file("business-licences.geojson")

gdf.head()
```

	folderyear	licencersn	licencenumber	licencerevisionnumber	businessname	businesstraden
0	25	4637506	25-129785	00	(Anie Philip)	N
1	25	4637507	25-129786	00	(Tony Haughian)	N
2	25	4637508	25-129787	00	(Tony Haughian)	N
3	25	4637511	25-129790	00	Simone Deborah Avram (Simone Avram)	N
4	25	4637516	25-129795	00	(William Squibb)	N

5 rows × 26 columns

```
print("Rows     Columns")
gdf.shape
```

```
Rows     Columns
(204107, 26)
```

```
gdf.columns
```

```
Index(['folderyear', 'licencersn', 'licencenumber', 'licencerevisionnumber',
       'businessname', 'businesstradename', 'status', 'issueddate',
       'expireddate', 'businesstype', 'businesssubtype', 'unit', 'unittype',
       'house', 'street', 'city', 'province', 'country', 'postalcode',
       'localarea', 'numberofemployees', 'feepaid', 'extractdate', 'geom',
```

```
       'geo_point_2d', 'geometry'],
      dtype='object')
```

```
gdf.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
RangeIndex: 204107 entries, 0 to 204106
Data columns (total 26 columns):
 #   Column                Non-Null Count   Dtype
---  ------                --------------   -----
 0   folderyear            204107 non-null  object
 1   licencersn            204107 non-null  object
 2   licencenumber         204107 non-null  object
 3   licencerevisionnumber  204107 non-null  object
 4   businessname          190635 non-null  object
 5   businesstradename     77307 non-null   object
 6   status                204107 non-null  object
 7   issueddate            175296 non-null  datetime64[ms, UTC]
 8   expireddate           175320 non-null  datetime64[ms]
 9   businesstype          204107 non-null  object
 10  businesssubtype       21518 non-null   object
 11  unit                  48914 non-null   object
 12  unittype              48650 non-null   object
 13  house                 109704 non-null  object
 14  street                109721 non-null  object
 15  city                  204051 non-null  object
 16  province              204027 non-null  object
 17  country               159233 non-null  object
 18  postalcode            108966 non-null  object
 19  localarea             201393 non-null  object
 20  numberofemployees     204107 non-null  float64
 21  feepaid               128649 non-null  float64
 22  extractdate           204107 non-null  datetime64[ms, UTC-07:00]
 23  geom                  0 non-null       object
 24  geo_point_2d          102566 non-null  object
 25  geometry              102566 non-null  geometry
dtypes: datetime64[ms, UTC-07:00](1), datetime64[ms, UTC](1), datetime64[ms](1), float64(2),
memory usage: 40.5+ MB
```

```
gdf.describe(include="all")
```

	folderyear	licencersn	licencenumber	licencerevisionnumber	businessname	businesst
count	204107	204107	204107	204107	190635	
unique	3	204100	198523	7	61773	
top	26	4983726	26-110595	00	Parking Corporation of Vancouver	Fasken Du
freq	71239	4	5	147350	503	
mean	NaN	NaN	NaN	NaN	NaN	
min	NaN	NaN	NaN	NaN	NaN	
25%	NaN	NaN	NaN	NaN	NaN	
50%	NaN	NaN	NaN	NaN	NaN	
75%	NaN	NaN	NaN	NaN	NaN	
max	NaN	NaN	NaN	NaN	NaN	
std	NaN	NaN	NaN	NaN	NaN	

11 rows × 26 columns

```
gdf.isnull().sum()
```

	0
folderyear	0
licencersn	0
licencenumber	0
licencerevisionnumber	0
businessname	13472
businesstradename	126800
status	0
issueddate	28811
expireddate	28787
businesstype	0
businesssubtype	182589
unit	155193
unittype	155457
house	94403
street	94386
city	56
province	80
country	44874
postalcode	95141
localarea	2714
numberofemployees	0
feepaid	75458
extractdate	0
geom	204107
geo_point_2d	101541
geometry	101541

dtype: int64

Missing Values

Geometry

```
gdf.geometry.isnull().sum()
```

```
np.int64(101541)
```

There are a lot of missing values containing no location. Businesses without any coordinates can't appear on a map or be geographically clustered. Because of this, I think it's reasonable to remove these data entries.

```
# Drop entries that have null values in the "geometry" column
gdf = gdf.dropna(subset=["geometry"])
```

This has removed a lot of entries, and I can see that there are significantly less null values than there were before.

```
gdf.isnull().sum()
```

	0
folderyear	0
licencersn	0
licencenumber	0
licencerevisionnumber	0
businessname	4
businesstradename	50744
status	0
issueddate	11674
expireddate	11655
businesstype	0
businesssubtype	82693
unit	57113
unittype	57355
house	0
street	0
city	0
province	0
country	11509
postalcode	731
localarea	0
numberofemployees	0
feepaid	37923
extractdate	0
geom	102566
geo_point_2d	0
geometry	0

dtype: int64

Number of Employees

```
gdf.numberofemployees.isnull().sum()
```

```
np.int64(0)
```

There are no missing values for number of employees, so I won't make any changes here.

Fee Paid

```
gdf.feepaid.isnull().sum()
```

```
np.int64(37923)
```

There are a lot of feepaid values missing. I'm going to decide how to handle these later during feature engineering because I'm not sure how to fill these in yet and can't just get rid of them since there's so many. I will likely do median imputation because it fills in values and is less swayed by outliers.

Filtering

Status: Active Businesses

```
gdf["status"].value_counts()
```

status	count
Issued	85872
Pending	5909
Gone Out of Business	4800
Inactive	4184
Cancelled	1801

dtype: int64

```
issued = gdf[gdf["status"] == "Issued"].copy()

print("Rows    Columns")
issued.shape
```

```
Rows    Columns
(85872, 26)
```

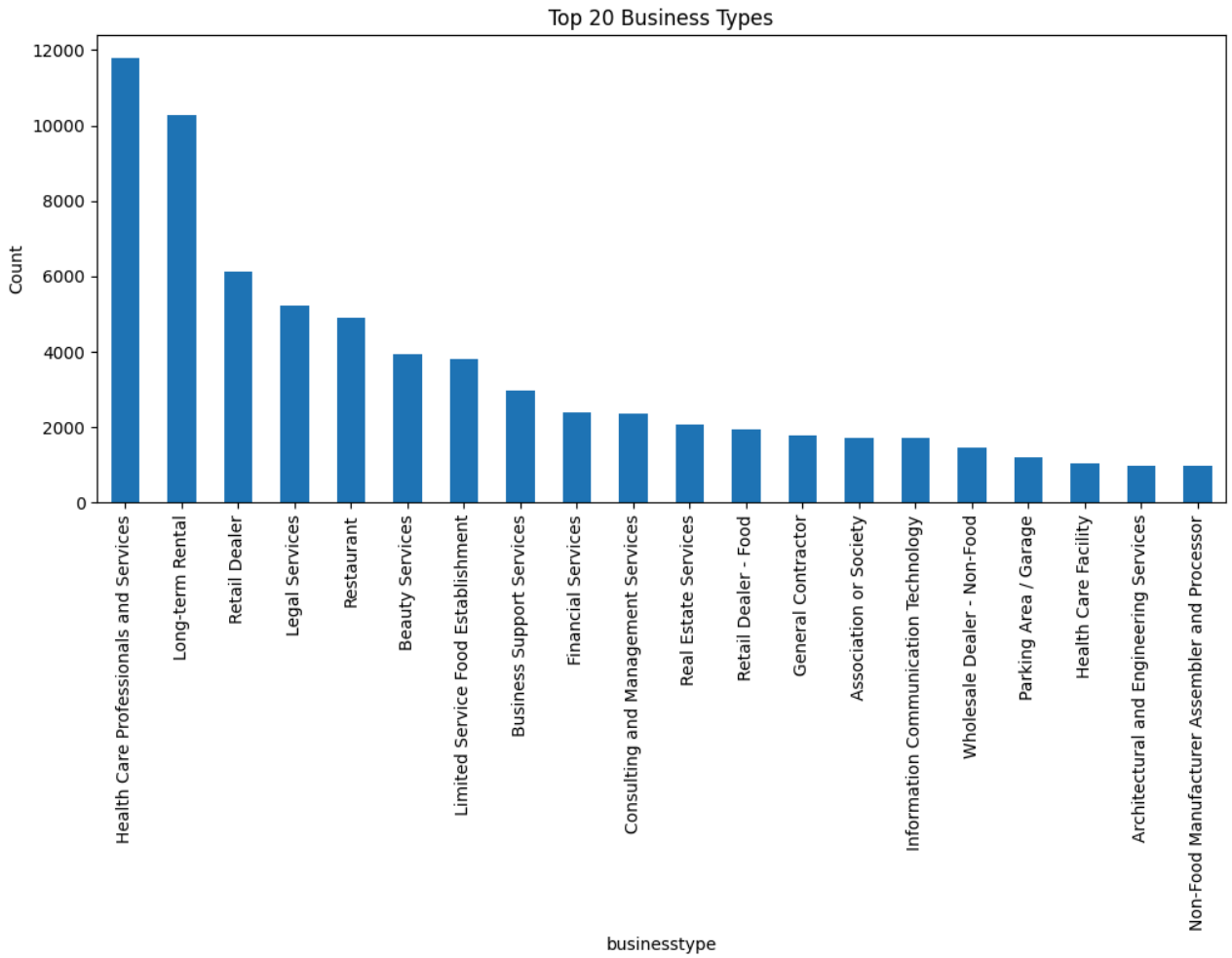
I chose to filter by status as used as an example in the assignment. I thought that filtering by status == "Issued" was a good idea because it allows us to only see active businesses that are currently licenced. If I'm wanting to understand the current business landscape, this makes the most sense. So, I don't really care about businesses that aren't active.

Business Type

```
business_type_count = issued["businesstype"].value_counts()
print(business_type_count)
```

```
businesstype
Health Care Professionals and Services    11803
Long-term Rental                          10266
Retail Dealer                              6137
Legal Services                             5211
Restaurant                                 4914
                                          ...
Urban Farm Class B                            4
Adult Services                                3
Amusement Park                                3
Oil Gas and Other Fuels                       3
Marine Service Station                        3
Name: count, Length: 90, dtype: int64
```

```
business_type_count.head(20).plot(kind = "bar", figsize=(12, 5))

plt.title("Top 20 Business Types")
plt.ylabel("Count")
plt.show()
```



My approach is to keep the most common categories, while putting businesses that are in rare categories into a new category called "other". This way, we can still see the most prevalent business types, and rare categories with very low prevalance are consolodated into one single category. It's likely that we're less interested in the low-prevalance categories too.

```
top_categories = business_type_count.head(15).index

issued["businesstype_grouped"] = issued ["businesstype"].where(
    issued["businesstype"].isin(top_categories),
    "Other"
)
```

```
issued["businesstype_grouped"].value_counts()
```

businesstype_grouped	count
Other	22863
Health Care Professionals and Services	11803
Long-term Rental	10266
Retail Dealer	6137
Legal Services	5211
Restaurant	4914
Beauty Services	3940
Limited Service Food Establishment	3814
Business Support Services	2961
Financial Services	2401
Consulting and Management Services	2353
Real Estate Services	2055
Retail Dealer - Food	1935
General Contractor	1789
Association or Society	1724
Information Communication Technology	1706

dtype: int64

Combining Business Type and Business Subtype

Some business types have a subtype, but not all. For the ones that do, I can combine them. This gives more detailed information about the businesses.

```
issued["industry"] = issued.apply(
    lambda row:
    f"{row["businesstype"]} - {row["businesssubtype"]}"
    if pd.notna(row["businesssubtype"])
    else row["businesstype"],
  axis=1
)
```

Reducing Sample Size

There are 26 columns of data. Not all of them are important and will make responsiveness in Streamlit difficult later. I'll be keeping only the columns I will actually use.

```
keep = [
  'businessname',
  'businesstradename',
  'licencerevisionnumber',
  'status',
  'issueddate',
  'expireddate',
  'businesstype',
  'postalcode',
  'localarea',
  'numberofemployees',
  'feepaid',
  'geo_point_2d',
  'geometry',
  'industry',
  'businesstype_grouped'
]

issued = issued[keep]
```

```
issued.info()
```

```
<class 'geopandas.geodataframe.GeoDataFrame'>
Index: 85872 entries, 1 to 203950
Data columns (total 15 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   businessname          85869 non-null  object
 1   businesstradename     42922 non-null  object
 2   licencerevisionnumber  85872 non-null  object
 3   status                85872 non-null  object
 4   issueddate            85726 non-null  datetime64[ms, UTC]
 5   expireddate           85725 non-null  datetime64[ms]
 6   businesstype          85872 non-null  object
 7   postalcode            85218 non-null  object
 8   localarea             85872 non-null  object
 9   numberofemployees     85872 non-null  float64
 10  feepaid               59392 non-null  float64
 11  geo_point_2d          85872 non-null  object
 12  geometry              85872 non-null  geometry
 13  industry              85872 non-null  object
```

```
 14  businesstype_grouped   85872 non-null  object
dtypes: datetime64[ms, UTC](1), datetime64[ms](1), float64(2), geometry(1), object(10)
memory usage: 10.5+ MB
```

```
issued.head()
```

	businessname	businesstradename	licencerevisionnumber	status	issueddate	expireddate
1	(Tony Haughian)	None	00	Issued	2024-11-18 17:52:13+00:00	2025-12-31
2	(Tony Haughian)	None	00	Issued	2024-11-18 17:52:11+00:00	2025-12-31
3	Simone Deborah Avram (Simone Avram)	None	00	Issued	2025-01-13 23:28:19+00:00	2025-12-31
4	(William Squibb)	None	00	Issued	2025-02-10 02:52:10+00:00	2025-12-31
5	(Daniel Dittrich)	None	00	Issued	2024-11-25 05:13:44+00:00	2025-12-31

```
issued.describe()
```

	expireddate	numberofemployees	feepaid
count	85725	85872.000000	59392.000000
mean	2025-12-30 06:16:17.385000	17.983324	663.150780
min	2024-01-12 00:00:00	0.000000	2.000000
25%	2024-12-31 00:00:00	1.000000	265.000000
50%	2025-12-31 00:00:00	3.000000	277.000000
75%	2026-12-31 00:00:00	9.000000	481.250000
max	2027-12-31 00:00:00	5876.000000	63722.000000
std	NaN	102.313693	1518.845538

Location Only Clustering

Latitude and Longitude

```
issued["longitude"] = issued.geometry.x
issued["latitude"] = issued.geometry.y

X = issued[["longitude", "latitude"]]
```

The geometry column contains points with an x and a y. X will store coordinates like this: (x, y).

```
scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)
```

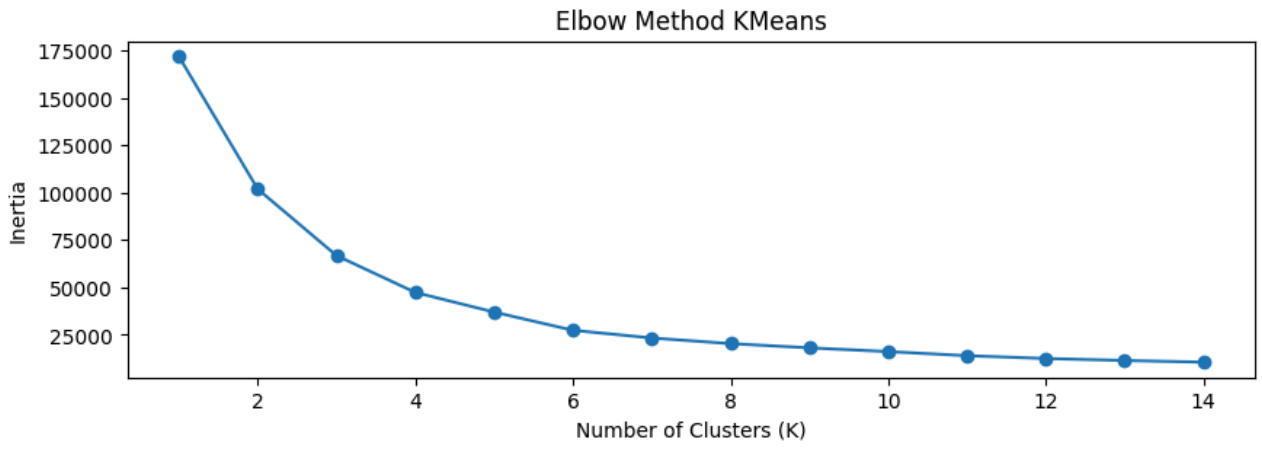
Latitude and longitude need to be scaled because they have different ranges. Standardizing means both x and y can have equal impact.

Elbow Method

```
inertia = []

K = range(1, 15)

for k in K:
  model = KMeans(
      n_clusters = k,
      random_state = 42,
      n_init = 10
  )

  model.fit(X_scaled)
  inertia.append(model.inertia_)
```

I need to choose the number of clusters for KMeans, so this lets us test different values.

```
plt.figure(figsize=(10,3))

plt.plot(K, inertia, marker="o")

plt.xlabel("Number of Clusters (K)")
plt.ylabel("Inertia")
plt.title("Elbow Method KMeans")

plt.show()
```



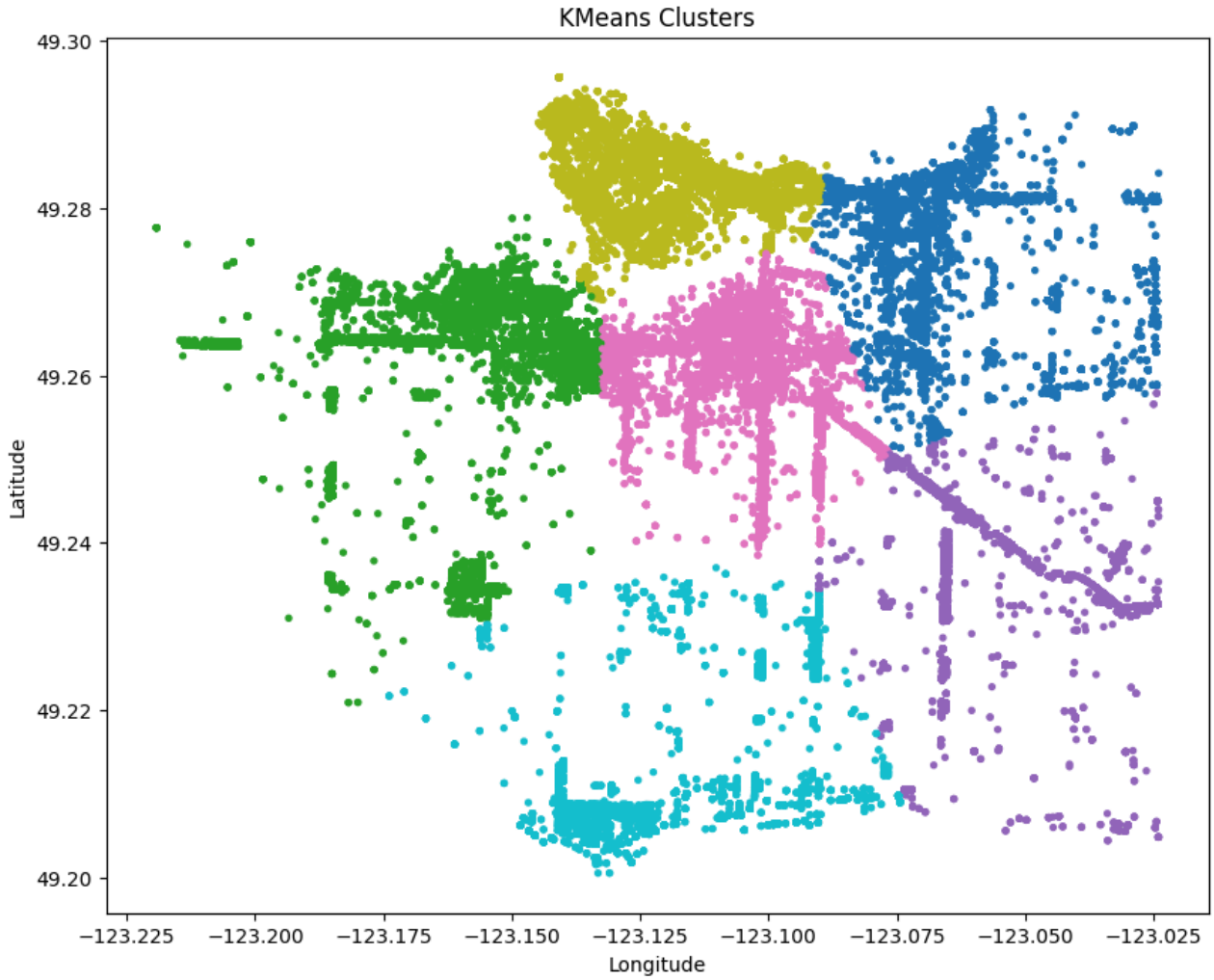
I *think* the elbow is at 6... I can't really tell if it should be 4 or 6 but I think I will stick with 6 because that's where it really flattens out.

Fitting

```
kmeans = KMeans(
    n_clusters = 6,
    random_state = 42,
    n_init = 10
)

issued["kmeans_cluster"] = kmeans.fit_predict(X_scaled)
```

The number of clusters is replaced with what the visualization above revealed.

```
plt.figure(figsize = (10, 8))

plt.scatter(
    issued["longitude"],
    issued["latitude"],
    c = issued["kmeans_cluster"],
    cmap = "tab10",
    s = 8
)

plt.title("KMeans Clusters")
plt.xlabel("Longitude")
plt.ylabel("Latitude")

plt.show()
```



Each colour represents a cluster, and businesses that are in the same KMeans cluster are the same colour.

DBSCAN

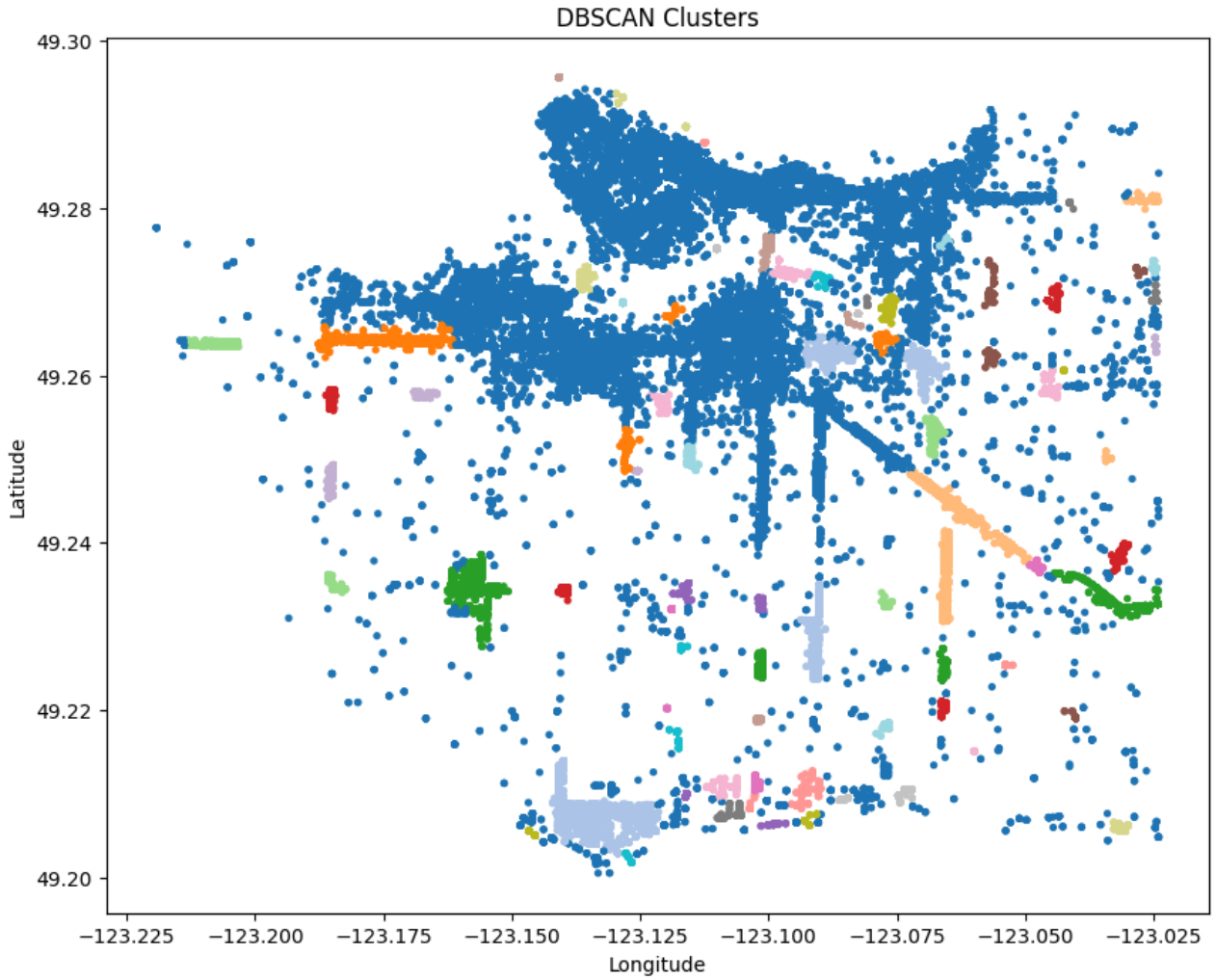
```
dbscan = DBSCAN(
    eps = 0.05,
    min_samples = 50
)

issued["dbscan_cluster"] = dbscan.fit_predict(X_scaled)
```

I don't need to choose k for DBSCAN, instead eps and min_samples. This takes some trial and error.

Eps is how close businesses must be to be considered neighbours. Smaller values mean more clusters, and larger values mean less clusters.

Min_samples means the minimum nearby businesses required to create a cluster. Larger values mean fewer clusters.

```
plt.figure(figsize = (10, 8))

plt.scatter(
    issued["longitude"],
    issued["latitude"],
    c = issued["dbscan_cluster"],
    cmap = "tab20",
    s = 8
)

plt.title("DBSCAN Clusters")
plt.xlabel("Longitude")
plt.ylabel("Latitude")

plt.show()
```



I experimented with A LOT of eps and min_samples values. However, this was about the best DBSCAN could do for clustering.

Counting Clusters

```
issued["kmeans_cluster"].value_counts().sort_index()
```

kmeans_cluster	count
0	9874
1	14005
2	5473
3	15883
4	33738
5	6899

dtype: int64

```
issued["dbscan_cluster"].value_counts().sort_index()
```

dbscan_cluster	count
-1	5341
0	1866
1	20522
2	38172
3	478
...	...
78	115
79	134
80	24
81	78
82	53

84 rows × 1 columns

dtype: int64

The KMeans clusters capture distinct areas of Vancouver, such as downtown, Mt. Pleasant, East Van, West side, South Vancouver, etc. We can see the downtown core very saturated with businesses, as it is very dense with points in the visualization (light green). Businesses become more sparse as they move away from the busiest areas of the city (downtown, west end, false creek). This map even allows us to see that businesses can be saturated around certain streets. Main street, Hastings, Kingsway, and W 4th are some of the streets that stand out. Since K-means divides the entire study area into a certain number of groups (I chose 6 here), every business is assigned to one cluster no matter what.

DBSCAN instead looks for dense concentrations of businesses. A very large cluster in the downtown core reaching out to the west side and part of east van has been identified because these businesses are closely packed together. Smaller areas with dense businesses have been identified as their own clusters. There has

also been a lot of noise identified, meaning these businesses don't belong to a distinct dense business district (because they aren't close enough to a dense area).

KMeans and DBSCAN both have correctly identified downtown as the densest business area of the city. They've also identified other areas where businesses are dense outside of downtown.

However, they do disagree in many areas. KMeans uses a defined number of clusters so even isolated businesses get sorted into one of the 6 clusters. DBSCAN leaves these isolated businesses as noise (-1). Since DBSCAN groups points based on their proximity, it produces irregular shaped clusters that can sometimes reflect real business areas better, however in this case I think KMeans does a better job. KMeans makes more equally sized clusters where every point belongs in a cluster.

Feature Engineering

Dates and Duration

```
# Converting these into datetime

issued["issueddate"] = pd.to_datetime(issued["issueddate"])
issued["expireddate"] = pd.to_datetime(issued["expireddate"])
```

```
# I'm getting the error that I can't subtract 2 different types of datetimes
# So I will make them both datetime naive since it's all in Vancouver

issued["issueddate"] = pd.to_datetime(issued["issueddate"]).dt.tz_localize(None)
issued["expireddate"] = pd.to_datetime(issued["expireddate"]).dt.tz_localize(None)
```

```
# Calculate licence duration (lifecycle)

issued["licence_duration"] = (
    issued["expireddate"] - issued["issueddate"]
).dt.days
```

Noticed some of the values in licence_duration were negative later on. Coming back here to check.

```
issued.loc[
    issued["licence_duration"] < 0,
    ["issueddate", "expireddate", "status", "licencerevisionnumber"]
].head(10)
```

	issueddate	expireddate	status	licencerevisionnumber
2720	2024-11-18 16:42:05	2024-11-16	Issued	00
2916	2026-02-03 17:51:28	2025-12-31	Issued	01
2976	2025-10-06 17:30:33	2024-12-31	Issued	01
4353	2025-01-06 23:03:31	2024-12-31	Issued	00
4470	2025-01-07 16:27:15	2024-12-31	Issued	00
4506	2025-01-02 16:24:34	2024-12-31	Issued	00
6883	2025-08-14 21:40:43	2025-08-14	Issued	00
7102	2026-01-08 21:34:20	2025-12-31	Issued	00
7163	2026-01-05 17:45:27	2025-12-31	Issued	00
11533	2024-05-10 22:12:26	2024-03-18	Issued	00

```
issued["licence_duration"] = issued["licence_duration"].abs()
```

Licence Revision Number

```
issued["licencerevisionnumber"].describe()
```

	licencerevisionnumber
count	85872
unique	7
top	00
freq	56628

dtype: object

```
issued["licencerevisionnumber"].value_counts()
```

licencerevisionnumber	count
00	56628
10	25287
01	2317
11	1497
02	86
12	53
03	4

dtype: int64

I Googled this and it looks like it refers to the number of revisions a licence has had. Will keep these all as integers.

```
issued["licencerevisionnumber"] = issued["licencerevisionnumber"].astype(int)
```

```
issued["licencerevisionnumber"].describe()
```

	licencerevisionnumber
count	85872.000000
mean	3.173025
std	4.648564
min	0.000000
25%	0.000000
50%	0.000000
75%	10.000000
max	12.000000

dtype: float64

Month Issued

```
# Convert this to datetime month

issued["issue_month"] = issued["issueddate"].dt.month
```

```
# For cyclic encoding, need to use sin and cos to make them like a circular clock

issued["month_sin"] = np.sin(
    2 * np.pi * issued["issue_month"] / 12
)

issued["month_cos"] = np.cos(
    2 * np.pi * issued["issue_month"] / 12
)
```

```
issued["month_sin"].describe()
```

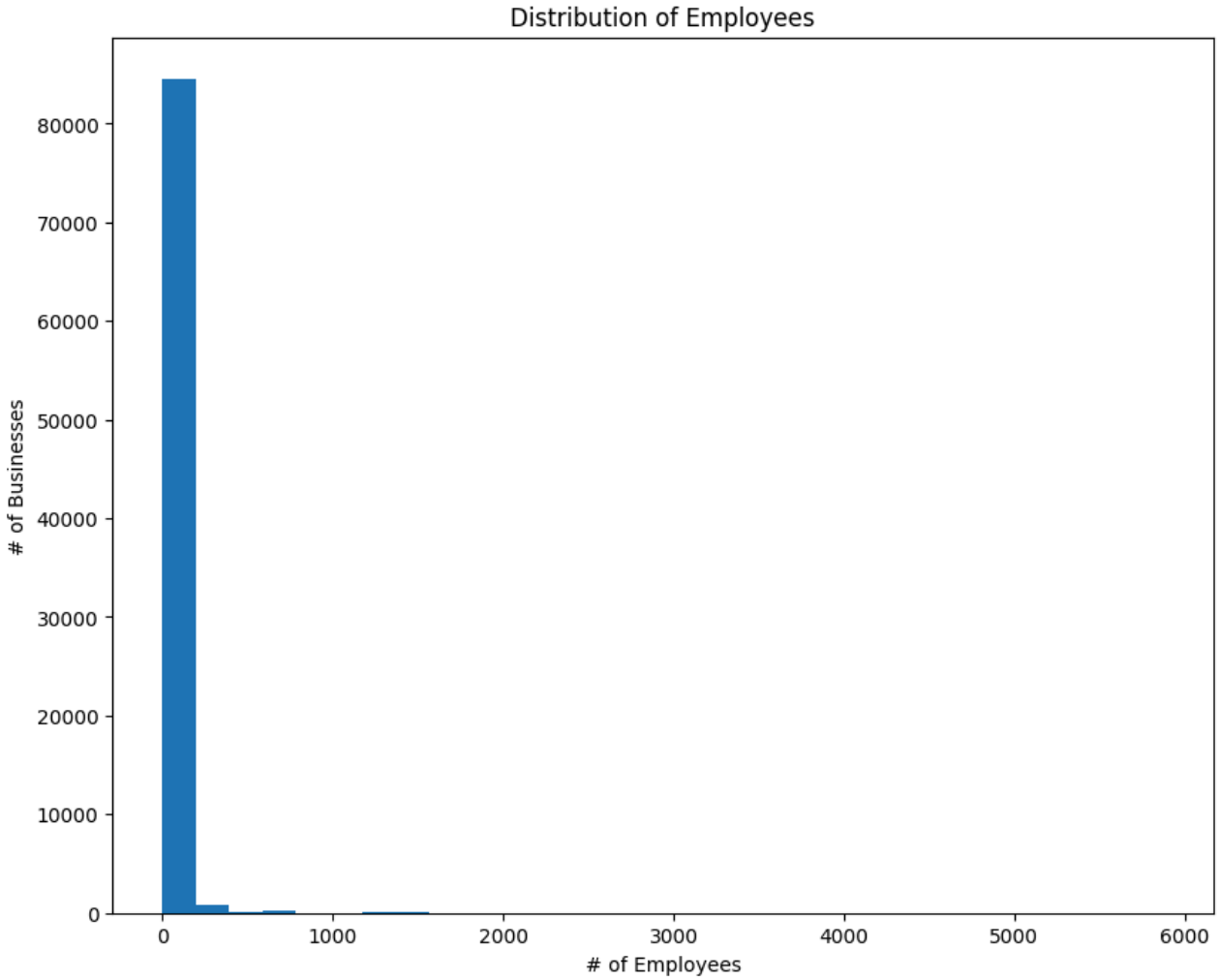
	month_sin
count	8.572600e+04
mean	3.929790e-03
std	4.926751e-01
min	-1.000000e+00
25%	-5.000000e-01
50%	-2.449294e-16
75%	5.000000e-01
max	1.000000e+00

dtype: float64

Since months repeat, need to have them appear in a cycle rather than a linear list of months that stop after December.

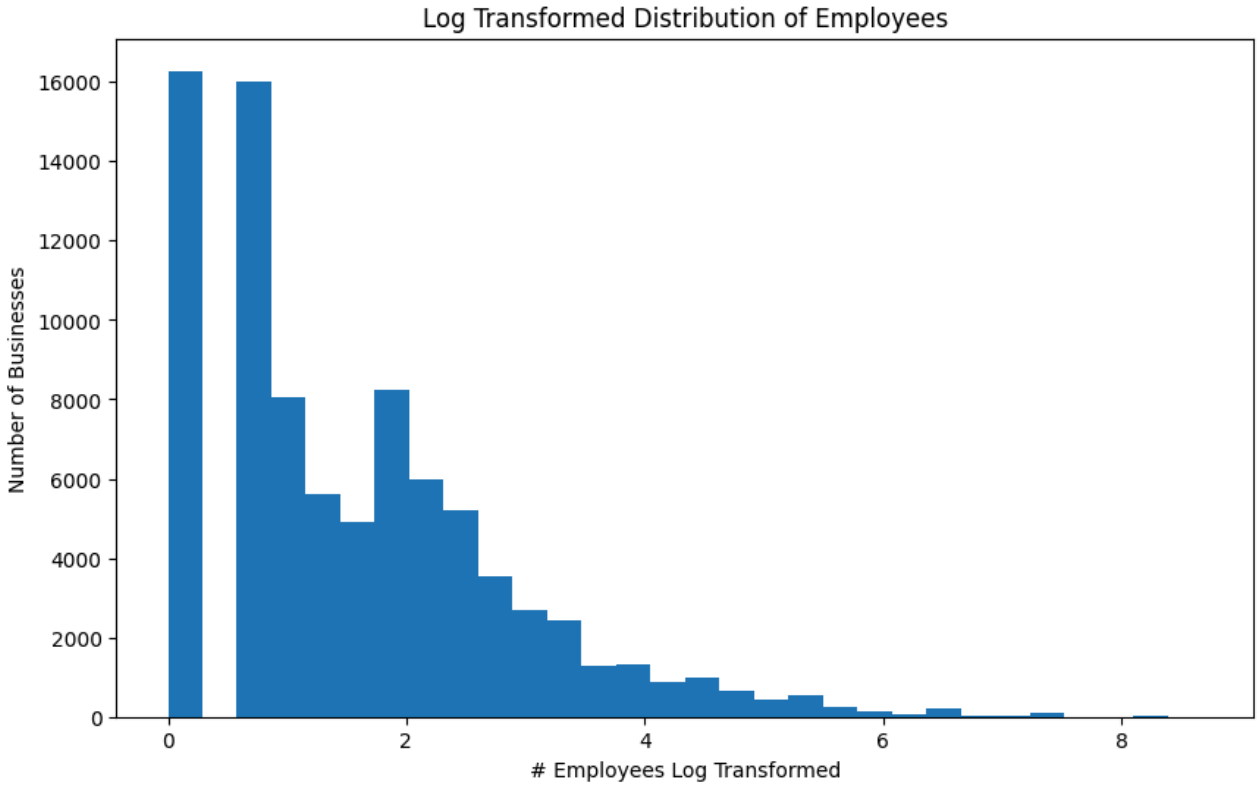
Number of Employees

```
plt.figure(figsize = (10, 8))

plt.hist(
    issued["numberofemployees"],
    bins = 30
)

plt.xlabel("# of Employees")
plt.ylabel("# of Businesses")
plt.title("Distribution of Employees")

plt.show()
```



After looking at this and the PCA graph, I realized that this data is extrememly left skewed, so I will log transform it.

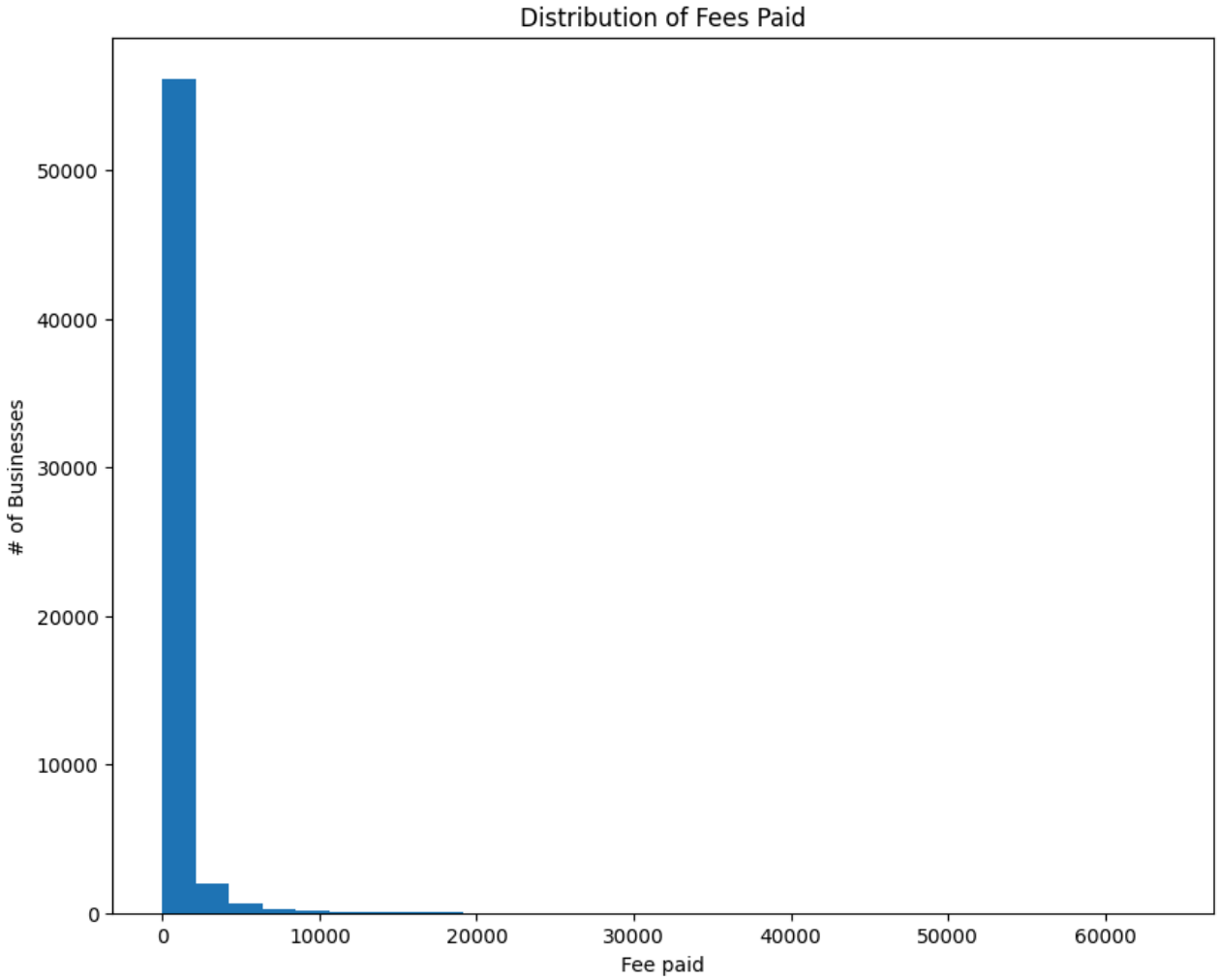
```
numberofemployees_log = np.log1p(issued["numberofemployees"])

plt.figure(figsize=(10, 6))

plt.hist(
    numberofemployees_log,
    bins=30
)

plt.xlabel("# Employees Log Transformed")
plt.ylabel("Number of Businesses")
plt.title("Log Transformed Distribution of Employees")

plt.show()
```




```
issued["numberofemployees_log"] = np.log1p(issued["numberofemployees"])
```

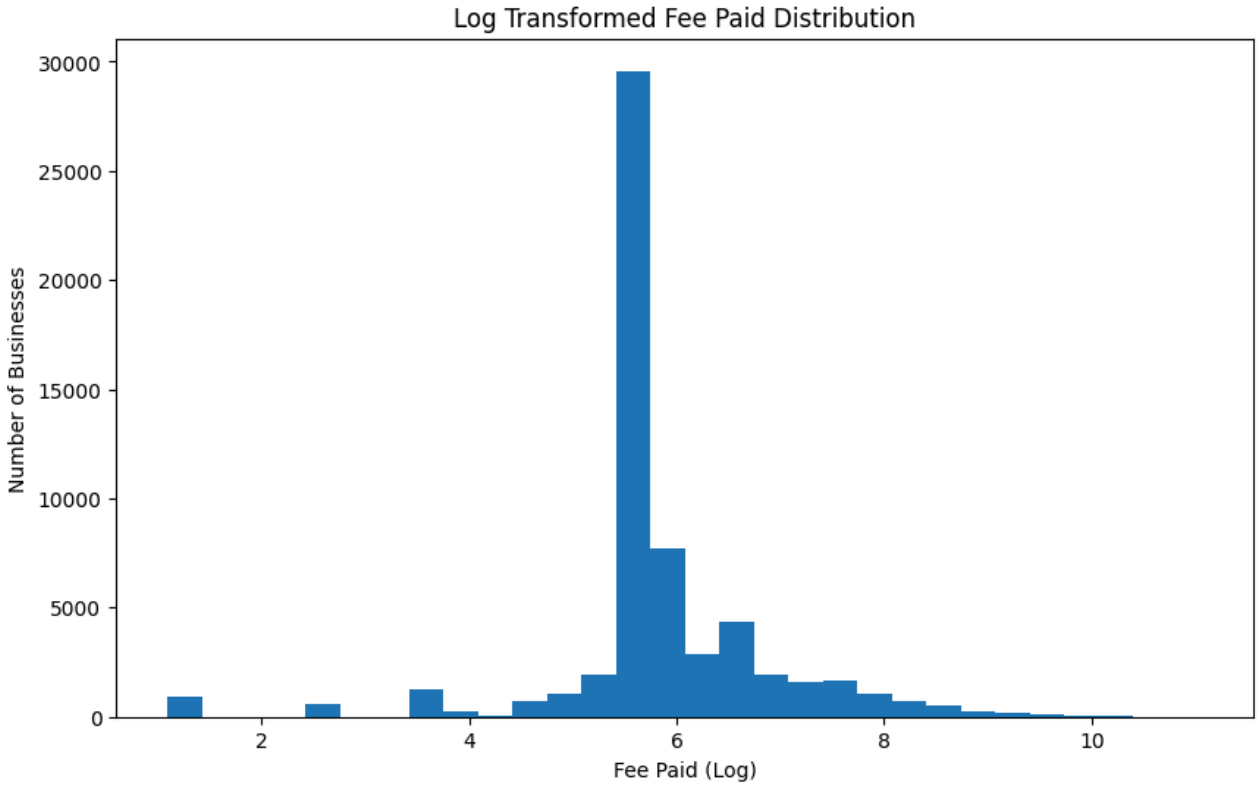
Fee Paid

```
plt.figure(figsize = (10, 8))

plt.hist(
    issued["feepaid"],
    bins = 30
)

plt.xlabel("Fee paid")
plt.ylabel("# of Businesses")
plt.title("Distribution of Fees Paid")

plt.show()
```



Same thing here. I will log transform feepaid so it's not quite as skewed.

```
feepaid_log = np.log1p(issued["feepaid"])

plt.figure(figsize=(10, 6))

plt.hist(
    feepaid_log,
    bins=30
)

plt.xlabel("Fee Paid (Log)")
plt.ylabel("Number of Businesses")
plt.title("Log Transformed Fee Paid Distribution")

plt.show()
```




```
issued["feepaid_log"] = np.log1p(issued["feepaid"])
```

Numeric Features

```
numeric_features = issued[[
    "numberofemployees_log",
    "feepaid_log",
    "licence_duration",
    "month_sin",
    "month_cos",
    "licencerevisionnumber"
]]
```

Encoding Business Categories

```
industry_encoded = pd.get_dummies(
    issued["businesstype_grouped"],
    prefix = "industry"
)

features = pd.concat(
    [numeric_features, industry_encoded],
    axis = 1
)

features.head()
```

	numberofemployees_log	feepaid_log	licence_duration	month_sin	month_cos	licencerevisio
1	0.0	5.424950	407.0	-0.500000	0.866025	
2	0.0	5.620401	407.0	-0.500000	0.866025	
3	0.0	5.241747	351.0	0.500000	0.866025	
4	0.0	6.082219	323.0	0.866025	0.500000	
5	0.0	5.645447	400.0	-0.500000	0.866025	

5 rows × 22 columns

Missing Values

```
print(issued['feepaid'].median())
```

```
277.0
```

```
issued["feepaid"].describe()
```

	feepaid
count	59392.000000
mean	663.150780
std	1518.845538
min	2.000000
25%	265.000000
50%	277.000000
75%	481.250000
max	63722.000000

dtype: float64

I'm choosing to do median imputation here because it's less affected by some of the outliers and extreme values in the dataset for feepaid, and all the other numeric values in the list of features.

```
features = features.fillna(
    features.median(numeric_only=True)
)
```

```
features.isnull().sum()
```

	0
numberofemployees_log	0
feepaid_log	0
licence_duration	0
month_sin	0
month_cos	0
licencerevisionnumber	0
industry_Association or Society	0
industry_Beauty Services	0
industry_Business Support Services	0
industry_Consulting and Management Services	0
industry_Financial Services	0
industry_General Contractor	0
industry_Health Care Professionals and Services	0
industry_Information Communication Technology	0
industry_Legal Services	0
industry_Limited Service Food Establishment	0
industry_Long-term Rental	0
industry_Other	0
industry_Real Estate Services	0
industry_Restaurant	0
industry_Retail Dealer	0
industry_Retail Dealer - Food	0

dtype: int64

No null values left.

Scaling Features

```
# I already have a scaler from earlier so I'll just use that

features_scaled = scaler.fit_transform(features)
```

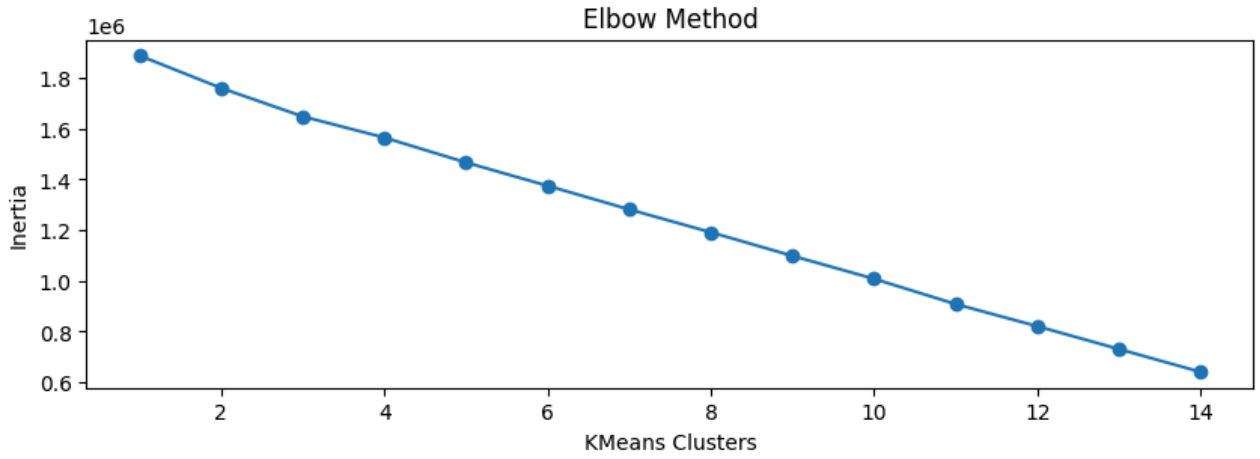
This helps make sure every feature has more "equal" input for model decision making.

KMeans Clustering

Elbow Method

```
inertia = []

K = range(1, 15)

for k in K:
  model = KMeans(
      n_clusters = k,
      random_state = 42,
      n_init = 10
  )

  model.fit(features_scaled)
  inertia.append(model.inertia_)
```

```
plt.figure(figsize = (10, 3))

plt.plot(K, inertia, marker = "o")

plt.xlabel("KMeans Clusters")
plt.ylabel("Inertia")
plt.title("Elbow Method")

plt.show()
```



This seems to only give 3 clusters.

```
kmeans = KMeans(
    n_clusters = 3,
    random_state = 42,
)

issued["feature_cluster"] = kmeans.fit_predict(features_scaled)
```

PCA

```
pca = PCA(n_components = 2)
```